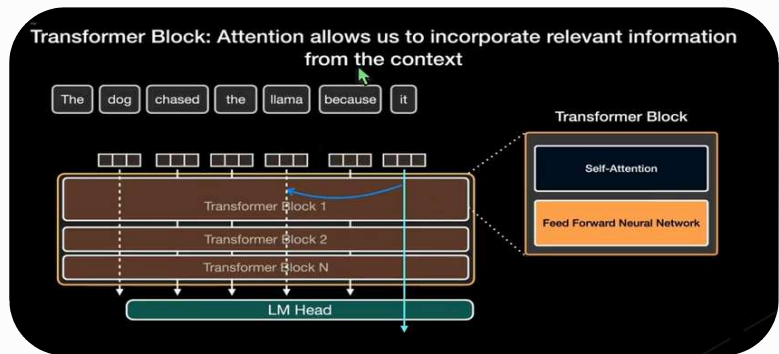




Transformer Block Explained | How Self-Attention and Feed-Forward Work

0:00

In this lecture, we'll focus on

0:02

understanding the transformer block. One

0:06

of the core components of the

0:07

transformer architecture. Let's take an

0:10

Show more ▾

The video gives a good high-level overview, but it skips many important details about **what actually happens inside a Transformer block**. Let's go through it step by step with the image you shared.

Big Picture

The sentence is:

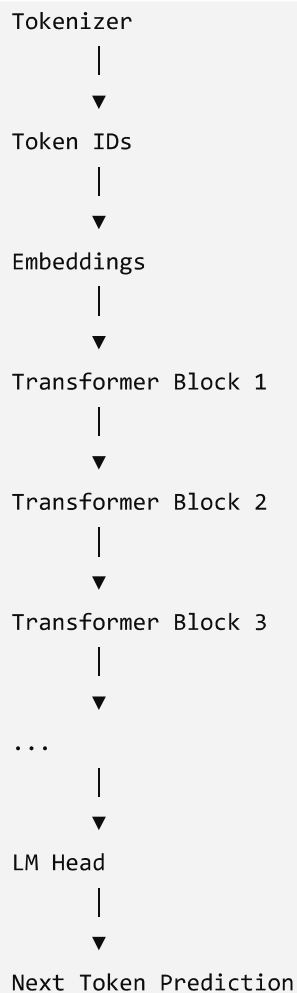
The dog chased the llama because it

The model wants to predict the next word.

The pipeline is:

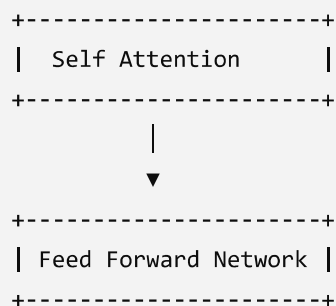
Input Text





Every transformer block performs **two major operations**:

Transformer Block



Step 1: Token Embeddings

Sentence

The dog chased the llama because it



Tokenizer converts it into tokens.

[The] [dog] [chased] [the] [llama] [because] [it]



Each token becomes a vector.

Example

"The"



[0.21
0.83
0.14
...
0.91]

GPT models usually use vectors of size

768
1024
2048
4096
8192



depending on the model.

Suppose embedding size = 768

7 words



↓

7 × 768 matrix

Step 2: Positional Encoding

Words alone aren't enough.

These two sentences have different meanings.

Dog bites man



Man bites dog

Embeddings are identical words.

Transformer adds position information.

Word	Position
The	0
dog	1
chased	2
the	3
llama	4
because	5
it	6



Final embedding

Embedding + Position



Step 3: Enter Transformer Block

Each transformer block performs

Input Embeddings



↓

Self Attention

↓

Feed Forward Network

↓

Output Embeddings

This happens 12, 24, 48, or even 96 times depending on model size.

Self Attention

This is the most important idea.

Each word asks

Which other words should I pay attention to?

Example

The dog chased the llama because it



Suppose we're processing

it



Question:

What does **it** refer to?

Possible answers

dog

or

llama



Self-attention computes which earlier tokens are most relevant.

Example attention scores

Word	Score
The	0.01
dog	0.18
chased	0.05
the	0.01
llama	0.65
because	0.08
it	0.02

The model learns that "**it**" most strongly relates to "**llama**" (or "dog" depending on context).

How Self Attention Works

Every token creates three vectors:

Query (Q)



Key (K)

Value (V)

For every word.

Example

dog



↓

Q

K

V

llama



↓

Q

K

V

it



↓

Q

K

V

Query

Query means

What information am I looking for?

Key

Key means

What information do I contain?

Value

Value means

If someone attends to me, what information should I provide?

Attention Score

The model computes

`Query(it)`

`×`

`Keys(all previous words)`



Like

`Q(it)`

`•`

`K(dog)`

`=`

`2.1`



`Q(it)`

`•`



$K(\text{llama})$

=

9.8

Larger score

↓

More attention

Then Softmax converts scores into probabilities.

Example

Before

2

5

7



After softmax

0.05

0.20

0.75



Now they sum to 1.

Weighted Sum

Each Value vector is multiplied by its attention weight.

Example

Dog

Weight = 0.2

Value = V_{dog}



Llama



Weight = 0.7

Value = Vllama

Final representation

$0.2 \times V_{dog}$



+

$0.7 \times V_{llama}$

+

...

Now "it" carries context from the relevant earlier words.

Multi-Head Attention

The model doesn't use just one attention mechanism.

It uses many in parallel.

Example

Head 1

Grammar



Head 2

Subject

Head 3

Verb

Head 4

Pronoun

Head 5

Long-distance dependency

...

GPT-3 has **96 attention heads**.

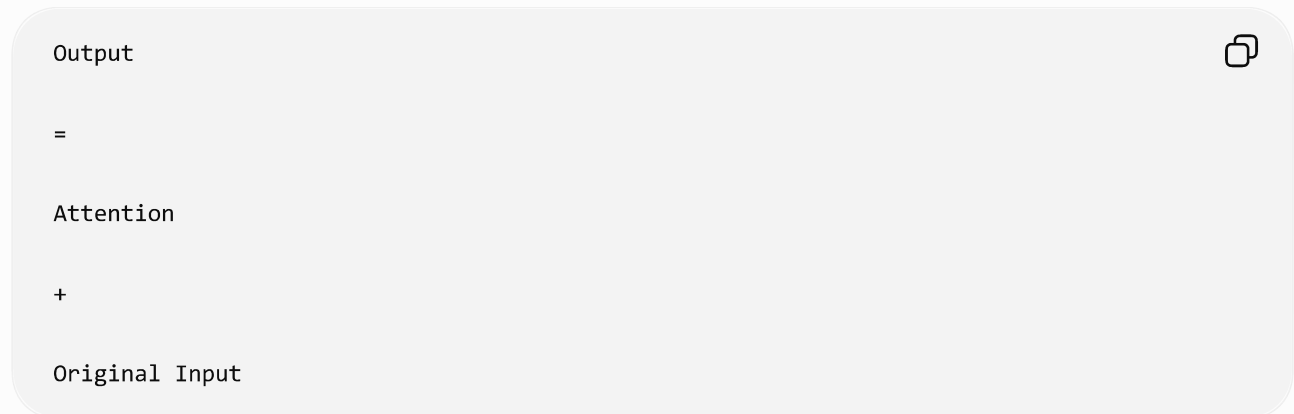
Each head focuses on different linguistic patterns.

Residual Connection

After attention

Instead of replacing the old embedding

Transformer adds it back.

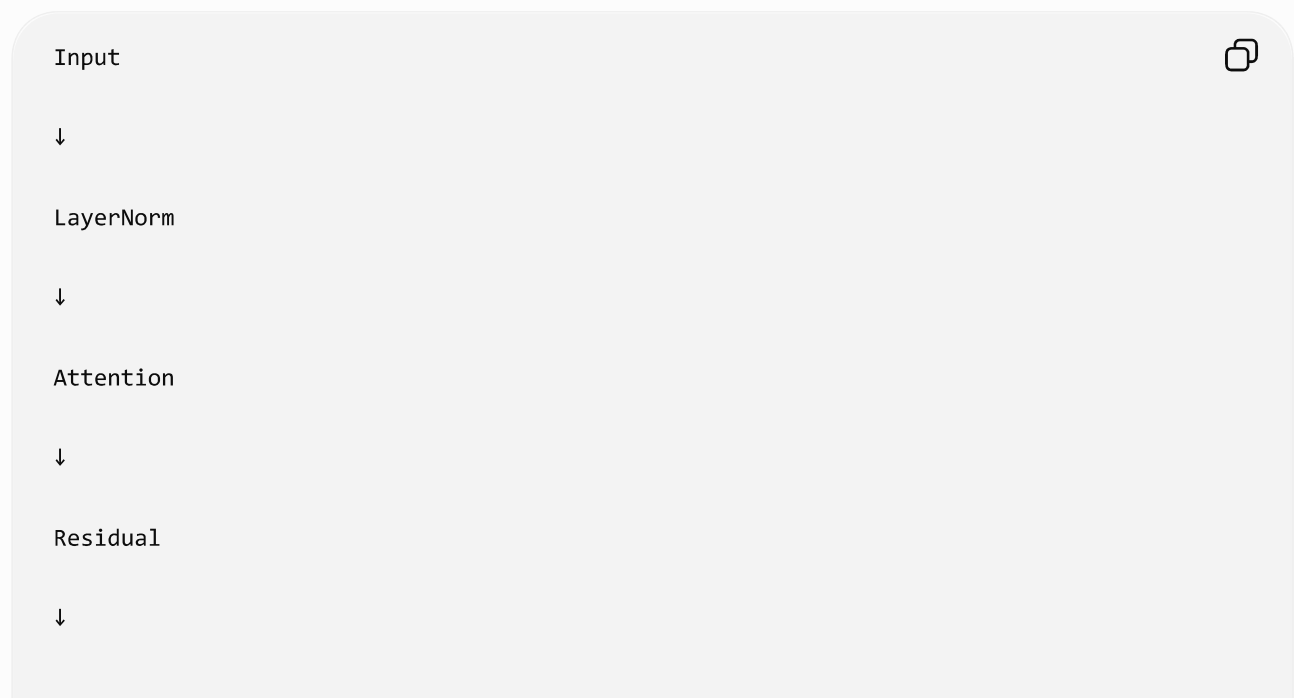


This helps preserve information and improves training stability.

Layer Normalization

The combined vectors are normalized.

Think of it like keeping values in a stable range so the network trains more reliably.



Feed Forward Network (FFN)

This is the second major component.

The video says

It extracts deeper features.

Let's see how.

Suppose after attention

Dog

↓

[0.21

0.84

...

]



FFN applies

Linear

↓

Activation (GeLU)

↓

Linear



Diagram:

768

↓

3072

↓



GeLU

↓

768

The first layer expands the feature space, the activation introduces non-linearity, and the second layer projects it back to the original size.

This lets the model learn richer transformations for each token independently.

Why Attention First?

Attention mixes information **between tokens**.

FFN processes **each token separately** after that contextual information has been incorporated.

So:

Attention



↓

Context

↓

Feed Forward

↓

Better Representation

Residual Again

The FFN output is also added back to its input.

FFN Output



+

Repeat Many Times

One block isn't enough.

Block 1



↓

Block 2

↓

Block 3

↓

...

↓

Block N

Each layer refines the token representations.

LM Head

After the final transformer block:

Hidden Vector



↓

Linear Layer

↓

Vocabulary Scores

↓

Softmax

↓

Next Token

If the vocabulary has 50,000 words, the LM head produces 50,000 scores—one for each possible next token.

Example:

Word	Probability
is	0.02
was	0.10
running	0.04
hungry	0.70
sleeping	0.14

The model picks (or samples) the next token.

Autoregressive Generation

Suppose input is

I



Predict

was



Now input becomes

I was



Predict

hungry



Now input becomes

I was hungry



Predict

today



This repeats until an end-of-sequence token or other stopping criterion is reached.

KV Cache

The video mentions caching.

Without a cache:

I



↓

Attention

↓

Compute

Next step

I was



↓

Recompute everything

Next

I was hungry



↓

Recompute everything

This is inefficient.

With a **Key-Value (KV) cache**:

Step 1

Store K,V for "I"

↓

Step 2

Reuse K,V for "I"

Only compute K,V for "was"

↓

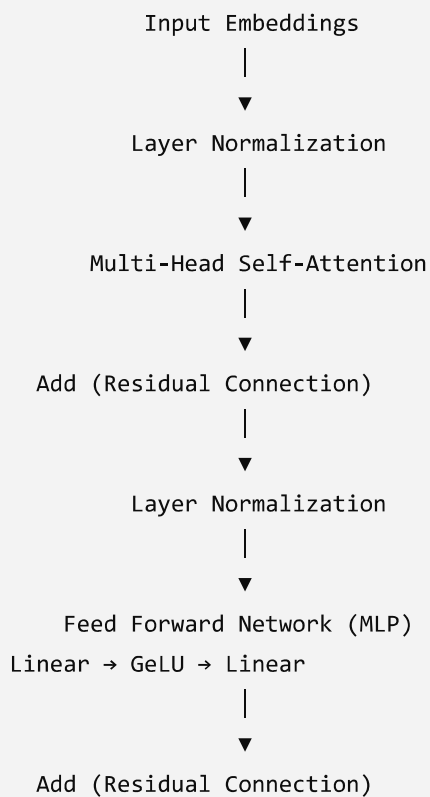
Step 3

Reuse cached K,V

Only compute K,V for "hungry"

Only the newest token needs fresh computation, which makes inference much faster.

Complete Transformer Block



↓
Output Embeddings

Summary

- **Self-attention** lets each token gather information from relevant tokens in the sequence using Query, Key, and Value vectors.
- **Multi-head attention** learns different types of relationships in parallel.
- **Feed-forward networks** apply non-linear transformations independently to each token after context has been mixed.
- **Residual connections** and **layer normalization** stabilize training and preserve information.
- Stacking many transformer blocks gradually builds richer semantic representations.
- The **LM head** converts the final hidden representations into probabilities over the vocabulary for next-token prediction.
- During generation, a **KV cache** reuses previously computed Keys and Values so only the newest token needs full attention computation.

